AI Image Generator

10. Performance

Version 2.0
22 October 2026

Project owner: Frazile AI
Platform: NVIDIA GeForce RTX 3090, Windows 11, Python 3.12

Z-Image Turbo, Stable Diffusion XL and its best fine-tunes, FLUX.1, LoRAs and Real-ESRGAN, served locally with Gradio in the Frazile AI studio UI, with per-image GPU telemetry recorded in SQLite and a live Admin panel.

10. Performance

All figures were measured on the development machine (RTX 3090, 24 GB, driver 610.88, torch 2.14.1 with CUDA 12.6) and taken from generations.db and the v2.0 model sweep.

10.1 Generation by model (832x1216, prompt weighting on for SDXL)

Model	Steps	CFG	Sampler	Inference	Peak VRAM
Z-Image Turbo	8	0	Flow-match Euler	8.6 s (9.8 s at 1344x768)	18.1 GB
Qwen-Image + Lightning (4-bit)	8	1 (none)	Flow-match Euler, exp. shift	13.4 s (13.3 to 14.2 s at 1344x768 and 1024x1024)	18.0 GB
SDXL base 1.0	30	7	DPM++ 2M Karras	8.0 s	13.1 GB
Juggernaut XL v9	30	5	DPM++ 2M Karras	8.0 s	13.1 GB
RealVisXL V5.0	30	4	DPM++ SDE Karras	8.0 s	13.2 GB
DreamShaper XL v2 Turbo	6	2	DPM++ SDE Karras	2.0 s	13.2 GB
Playground v2.5	40	3	EDM DPM-Solver	10.5 s	13.2 GB
Animagine XL 3.1	28	6	Euler a	7.3 s	13.2 GB

Every SDXL-family model runs at about 3.7 to 3.8 steps per second at this size. Inference time is therefore proportional to the step count, which is why the turbo model's 6 steps finish in 2 seconds. Z-Image Turbo runs at about 1.1 s per step but needs only 8, so a finished image takes 9 to 10 s with better prompt adherence and legible text. Loading an SDXL-family model from the local cache takes 26 to 27 s; freeing it returns resident VRAM to 0.01 GB. Measurements taken while another process holds GPU memory are not valid: with the web app resident alongside the test, Z-Image took 78 to 128 s per image because Windows paged VRAM.

10.2 Generation (v1.0 measurements, 1024x1024 and 16:9, SDXL base)

Size	Steps	Inference	Steps/s	Peak VRAM	Temp after	Power after
1024x1024	15	4.7 to 4.9 s	3.1 to 3.2	10.5 GB	55 to 56 C	377 to 379 W
1024x1024	30	9.7 s	3.1	10.5 GB	-	-
1344x768	40	12.8 to 13.5 s	3.0 to 3.1	10.4 GB	52 to 54 C	349 to 357 W
768x1344	40	11.2 to 11.4 s	3.5 to 3.6	10.5 GB	55 to 57 C	382 to 385 W

10.3 Upscaling

From	To	Passes	Time	Output file
1344x768	3840x2194 (4K)	1	3.4 s	9.9 MB
768x1344	2194x3840 (4K)	1	2.9 s	7.9 MB
768x1344	1463x2560 (2K, Upscale bar)	1	3.6 s	3.5 MB
768x1344	4388x7680 (8K)	2	8.4 to 8.5 s	30.5 MB

10.4 Memory

Stage	VRAM
Idle with one SDXL-family model loaded	6.6 GB allocated
Idle with Z-Image Turbo loaded (bf16 transformer + 8-bit text encoder)	15.8 GB allocated

Stage	VRAM
Idle with Qwen-Image loaded (NF4 transformer + 4-bit text encoder + Lightning LoRA)	17.5 GB allocated
Peak during Qwen-Image generation at 1 MP (VAE tiling on)	18.0 GB; 21.7 GB before VAE tiling, which caused paging
Peak during Z-Image generation at 1344x768	18.1 GB
Peak during 1 MP generation, plain prompt	10.4 to 10.5 GB
Peak during 832x1216 generation with compel embeddings	13.1 to 13.2 GB
After unload	0.01 GB
Upscaler weights	67 MB (half precision)
Peak during tiled upscale	no measurable increase over the generation peak

10.5 Startup and switching

Step	Time
First ever start (downloads 6.7 GB)	bandwidth dependent; the five fine-tunes downloaded in 76 to 184 s each on the development connection
SDXL load from local cache at start	34 to 59 s (disk cache dependent)
Z-Image Turbo first load (fp32 to bf16 conversion plus writing the 12 GB cache)	about 250 s
Z-Image Turbo load from models/cache/	about 130 s (8-bit quantisation of the text encoder dominates)
Qwen-Image first load (quantising 41 GB to NF4 and writing the 11 GB cache)	about 260 s; needs most of the 16 GB system RAM
Qwen-Image load from models/cache/	about 90 s
Model switch while running (SDXL family)	26 to 27 s
Upscaler load	under 1 s
Admin panel refresh	about 3 s first, 0.7 s after

10.6 Tuning levers

- Model: Z-Image Turbo gives a finished photoreal image in 9 s; DreamShaper Turbo gives a rougher draft in 2 s. For SDXL-specific looks or LoRAs, regenerate the chosen seed with Juggernaut or RealVis.
- Steps: the dominant cost for every model. `quick` halves them; `highly detailed` adds a third.
- Count: images in a batch are generated sequentially, so time scales linearly.
- Upscale later: the Upscale bar costs 3 to 9 s per image and avoids regenerating.
- Cards with less VRAM: generate.py enables CPU offload below 12 GB, which roughly doubles inference time but keeps the pipeline working.